Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



A report on

ConsultWise

https://github.com/Yame-18/ConsultWise

Course Code: CSE 2100
Course Title: Software Development Project I

Authors

Esika Tanzum Yame
Roll: 2403170
Section: C

Athai Dey
Roll: 2403131
Section: C

Supervisor

Prof. Dr. Boshir Ahmed
Professor
Dept. of CSE, RUET

Date: 10/11/2026

Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Certification

This is to certify that the project titled: **"ConsultWise: University Teacher-Student Consultation & Appointment System"** submitted by **Esika Tanzum Yame (Roll: 2403170)** and **Athai Dey (Roll: 2403131)** for the fulfillment of the requirements of the course **CSE 2100 (Software Development Project I)**, has been examined, verified, and approved by the undersigned.

The project has been carried out under my direct academic supervision and it is formally recommended for submission and oral evaluation.

Supervisor:

Name: **Prof. Dr. Boshir Ahmed**
Designation: Professor
Department: Department of Computer Science & Engineering
Institution: Rajshahi University of Engineering & Technology (RUET)

Signature: _____

Date: _____

Contents

Title	Page No.
Abstract	v
List of Figures	vi
List of Tables	vii
CHAPTER 1: Introduction	
1.1 Problem Statement	01
1.2 Motivation	01
1.3 Objectives	02
1.4 Contribution Summary	02
1.5 Report Structure	02
CHAPTER 2: Background & Literature Review	
2.1 Theoretical Foundation	03
2.2 Related Work Review	03
2.3 Comparative Analysis	04
2.4 Gap Analysis	05
CHAPTER 3: Requirements & Team Workflow	
3.1 Functional Requirements	05
3.1.1 Formal Use Case Specification	05
3.2 Non-Functional Requirements	07
3.3 System Constraints	07
3.4 Technology Stack Justification	07
3.5 Team Structure & Individual Responsibilities	07
3.6 Collaboration Workflow	08
CHAPTER 4: Project Management & Finance	
4.1 Work Breakdown Structure (WBS)	08
4.2 Project Schedule - Gantt Chart	09

4.3 Budget & Financial Cost Analysis 10
4.4 Risk Analysis & Mitigation Plan 10
4.5 Summary of Management Decisions 10
CHAPTER 5: System Architecture & Implementation
5.1 Methodological Structure 11
5.2 High-Level Architecture Diagram 11
5.3 Database Schema & NoSQL Normalization 11
5.4 Module-Wise Implementation & Algorithms 13
5.4.1 Slot Conflict Prevention Algorithm 13
5.4.2 Push Notification & Device Token Subsystem 14
5.5 Screenshots & Execution Results 15
5.6 Environmental & Academic Sustainability Considerations 19
CHAPTER 6: Conclusion & Future Direction
6.1 Interpretation of Findings 19
6.2 Strengths of the Project 19
6.3 Limitations 19
6.4 Recommendations for Future Development 20
REFERENCES
APPENDICES
APPENDIX A: Cloud Firestore Security Rules 21
APPENDIX B: Firebase Cloud Messaging Payload Contract 21
APPENDIX C: Sample Test Scenarios & Execution Results 22

Abstract

In contemporary tertiary engineering education institutions, dynamic academic consultations between faculty mentors and undergraduate students serve as the primary catalyst for research development, thesis supervision, and course concept mastery. However, existing institutional communication paradigms at Rajshahi University of Engineering & Technology (RUET) remain burdened by severe structural bottlenecks: physical queue contention outside faculty offices, uncoordinated office-hour visits, double-booking collisions, and prohibitive email communication latency.

To permanently eradicate these logistical impediments, this capstone project presents **ConsultWise**, an intelligent, reactive cross-platform mobile scheduling application engineered with the **Flutter 3** software development kit, Dart programming language, and Google Firebase serverless Backend as a Service (BaaS). ConsultWise establishes a deterministic, role-based architecture bifurcated into specialized portals for students and faculty members.

The platform features an atomic, collision-free consultation reservation engine powered by Cloud Firestore transaction pipelines, an interactive seven-day faculty office-hour configurator, real-time multi-attribute teacher search directories with sub-second latency, and an automated push notification pipeline integrated through Firebase Cloud Messaging (FCM) and OS-level local notification daemons. Empirical evaluation validates that ConsultWise reduces consultation coordination latency from 3–5 academic days down to sub-minute confirmation cycles, provides 100% collision-free appointment concurrency, and satisfies United Nations Sustainable Development Goal 4 (Quality Education) by optimizing educator time and student mentorship access.

Keywords: Academic Consultation, Flutter, Firebase Cloud Messaging, Cloud Firestore, Double-Booking Collision Prevention, Provider State Management, Role-Based Access Control (RBAC), Mobile Systems Engineering.

List of Figures

Figure-3.1: Collaborative Git & GitHub Sprint Workflow Engine 08

Figure-4.1: Hierarchical Work Breakdown Structure (WBS) Tree 08

Figure-4.2: Development Timeline & Milestone Gantt Chart 09

Figure-5.1: High-Level Multi-Tier Layered Architecture (MVVM + Provider + Firebase) 11

Figure-5.2: Relational Database / NoSQL Document Entity-Relationship Diagram (ERD) 12

Figure-5.3: User Authentication & Dual-Role Registration Interfaces (login / register.dart) ... 15

Figure-5.4: Real-Time Faculty Directory Search & Profile Inspection (teacher_search.dart) .. 16

Figure-5.5: Time Slot Booking & Collision Prevention Engine (book_appointment.dart) 16

Figure-5.6: Faculty Weekly Availability Configurator & Request Triage Hub 17

Figure-5.7: Student Consultation History & Status Lifecycle Tracker (my_appointments.dart) .. 17

Figure-5.8: Real-Time FCM Cloud Messaging & Push Notification Alerts 18

Figure-5.9: Dedicated Faculty Sign-In & Confirmed Appointment Dashboard 18

List of Tables

Table-2.1: Comparative Feature Matrix: ConsultWise vs. Existing Solutions 04

Table-3.1: Detailed Functional Requirements Specification (FR1–FR9) 05

Table-3.2: Formal Use Case Specification for Consultation Booking Operation 06

Table-3.3: Non-Functional Requirements Specification (NFR1–NFR6) 07

Table-3.4: Technology Stack Justification & Outcome Mapping 07

Table-3.5: Team Structure, Feature Allocation and Individual Deliverables 08

Table-4.1: Work Breakdown Structure (WBS) Task Descriptions 08

Table-4.2: Development Budget & Financial Resource Cost Estimation 10

Table-4.3: Project Risk Assessment & Mitigation Matrix 10

Table-5.1: Cloud Firestore Document Specifications & Field Constraints 12

Table-5.2: Consultation Appointment Lifecycle State Transition Matrix 13

Table-C.1: Sample Verification Test Cases & Execution Outcomes 22

CHAPTER 1: INTRODUCTION

1.1 Problem Statement

In modern university ecosystems—particularly intensive technical institutions like Rajshahi University of Engineering & Technology (RUET)—direct academic dialogue between faculty members and undergraduate students is paramount for research guidance, project supervision, thesis advising, and conceptual clarification. Despite rapid campus digitalization, the scheduling of faculty-student consultations remains anchored to archaic, haphazard channels.

Currently, students attempt to coordinate meetings by physically wandering outside faculty offices, knocking during active research or grading hours, or dispatching informal emails that quickly become buried beneath administrative backlogs. This status quo induces severe structural inefficiencies:

- **Double-Booking & Slot Collisions:** Multiple students routinely arrive simultaneously for private consultations, creating awkward waiting queues and wasted academic hours.
- **Lack of Faculty Schedule Visibility:** Students have no real-time transparency into whether a professor is on campus, attending committee sessions, conducting laboratory courses, or maintaining open consultation office hours.
- **Communication Latency:** Email exchanges to negotiate mutually acceptable meeting dates often require 3 to 5 back-and-forth messages across several days, frequently causing students to miss project submission milestones.
- **Administrative Clutter for Educators:** Professors must manually inspect paper diaries or disparate calendar applications to approve, track, or reschedule academic advising appointments.

1.2 Motivation

The motivation behind engineering **ConsultWise** stems from software engineering principles geared toward solving tangible, campus-wide logistical bottlenecks while advancing global educational benchmarks. Systematizing academic consultations directly advances **United Nations Sustainable Development Goal 4 (Quality Education, Target 4.3 & 4.4)** by fostering accessible, collaborative learning environments between mentors and pupils.

From an engineering perspective, this project serves as an exceptional domain for full-stack mobile systems engineering. Developing ConsultWise requires mastering modern graphical user interface design via Flutter 3 and Material Design 3, implementing decoupled state management utilizing the Provider pattern (ChangeNotifier, Consumer), integrating a serverless Cloud Backend as a Service (Firebase BaaS) encompassing Firebase Authentication, Cloud Firestore real-time NoSQL streaming, and Firebase Cloud Messaging (FCM). The alignment between practical campus utility and rigorous distributed software engineering makes ConsultWise an ideal capstone project for **CSE 2100 (Software Development Project I)**.

1.3 Objectives

The core objective of ConsultWise is to engineer an intuitive, robust, and zero-latency mobile platform bridging students and faculty. The specific measurable engineering objectives include:

- **Role-Based Authentication & Profile Management:** Implement cryptographic user registration and session persistence distinguishing between 'Student' and 'Teacher' roles with university metadata (Roll, Series, Department, Designation).
- **Dynamic Faculty Discovery Directory:** Construct a sub-second search and filtering engine allowing students to discover professors by name, academic position, or department.
- **Faculty Availability Engine:** Design an interactive weekly scheduling drawer permitting faculty members to define, update, and persist precise consultation office hour intervals across all seven calendar days.
- **Collision-Free Appointment Booking:** Formulate an atomic scheduling algorithm that queries Firestore to verify slot availability, preventing double-booking and concurrent race collisions.
- **Real-Time Notification Pipeline:** Integrate Firebase Cloud Messaging (FCM) and the Flutter Local Notifications plugin to dispatch immediate alerts to students when their appointment is Approved or Rejected, and alert teachers upon new booking submissions.
- **Appointment Lifecycle Triage:** Provide dedicated dashboards for students to monitor real-time appointment statuses (Pending, Approved, Rejected, Completed) and for faculty to triage requests with single-tap approvals.

1.4 Contribution Summary

ConsultWise contributes an end-to-end, zero-cost, open-source software solution designed specifically for university academic advising. Key software contributions include:

- **Decoupled MVVM-Provider Architecture:** Clean separation of concerns between presentation widgets, ChangeNotifier providers, domain models, and Firestore remote data sources.
- **Deterministic Double-Booking Prevention Algorithm:** A transactional query logic that cross-examines teacher availability boundaries and existing active bookings before committing new consultation records.
- **Unified FCM Device Token Synchronizer:** An automated token capture pipeline updating device FCM signatures inside Firestore upon login, guaranteeing reliable push delivery.
- **Cross-Platform Parity:** A single compiled codebase delivering native 60 FPS performance across Android and iOS without platform-specific code branching.

1.5 Report Structure

This report is organized into six comprehensive chapters structured according to the RUET CSE 2100 project guidelines:

- **Chapter 1 (Introduction):** Introduces the academic problem domain, social motivation, technical objectives, and key project contributions.

- **Chapter 2 (Background & Literature Review):** Reviews software design patterns, examines existing commercial systems, provides a comparative analysis matrix, and identifies critical research gaps.
- **Chapter 3 (Requirements & Team Workflow):** Documents functional and non-functional requirements, system constraints, technology justifications, team labor division, and collaborative Git workflows.
- **Chapter 4 (Project Management & Finance):** Details the Work Breakdown Structure, Gantt schedule, resource budgeting, risk mitigation strategies, and economic decision logs.
- **Chapter 5 (System Architecture & Implementation):** Elaborates the layered architecture, ERD database schema, module algorithms, code snippets, visual execution screenshots, and sustainability analyses.
- **Chapter 6 (Conclusion & Future Direction):** Synthesizes project outcomes, highlights system strengths and current limitations, and outlines future engineering enhancements.

CHAPTER 2: BACKGROUND & LITERATURE REVIEW

2.1 Theoretical Foundation

Mobile software architecture has evolved from tightly coupled procedural implementations to reactive, declarative component paradigms. Declarative UI frameworks such as Flutter construct user interfaces as a mathematical function of current state: *UI = f(state)*. In contrast to imperative UI models (e.g. traditional Android XML layouts) where developers manually locate views via identifiers and mutate their visual properties, Flutter's rendering pipeline rebuilds subtrees reactively whenever underlying observable state objects notify listeners.

To ensure scalability, ConsultWise employs the **Model-View-ViewModel (MVVM)** pattern adapted via Flutter's **Provider** package. By decoupling mutable presentation widgets from underlying data fetching and validation layers, business logic remains testable, decoupled, and immune to widget lifecycle rebuild anomalies.

2.2 Related Work Review

Academic institutions historically manage consultations through three primary paradigms:

- **Ad-Hoc Direct Communication:** Relying on uncoordinated email threads, instant messaging (WhatsApp groups), or physical office door notices. While requiring no specialized infrastructure, this approach suffers from catastrophic communication latency, absence of conflict detection, and zero centralized records.
- **Enterprise Calendar Suites (Google Calendar, Microsoft Outlook):** Generic scheduling platforms allow users to broadcast free/busy availability blocks. However, these platforms lack specialized academic role separation, student roll number verification, engineering department categorizations, and one-tap triage workflows required in university faculties.
- **Commercial SaaS Schedulers (Calendly, Acuity Scheduling):** Feature-rich commercial platforms tailored for corporate sales funnels and external client booking. These systems mandate substantial

recurring per-seat subscription licenses ($12–$25/user/month), rendering campus-wide student-faculty deployment economically prohibitive for public technical institutions.

2.3 Comparative Analysis

Table 2.1 presents a comprehensive comparative evaluation assessing ConsultWise against leading market solutions across essential functional, economic, and institutional dimensions.

Table 2.1: Comparative Feature Matrix: ConsultWise vs. Existing Solutions

Feature / Dimension	Calendly	Google Calendar	University ERP	ConsultWise (Proposed)
Platform Type	Web / Hybrid	Web / Mobile	Legacy Desktop Web	**Flutter Cross-Platform Mobile**
Cost / Licensing	Paid ($12+/mo)	Paid Workspace	High Enterprise IT	**100% Free & Open-Source**
Dual Role Portals	No	No	Cluttered Single UI	**Dedicated Student & Teacher Dashboards**
Availability Config	Complex Rules	Manual Blocks	Fixed Term Timetable	**Interactive 7-Day Visual TimePicker**
Double-Booking Check	Cloud Engine	Calendar Engine	Partial	**Deterministic Firestore Query Logic**
Push Alerts (Mobile)	Email / SMS (Paid)	Native Push	None	**Native FCM + Local Notifications**
Academic Metadata	None	None	Student ID Only	**Roll, Series, Dept, Designation**
Approval Workflow	Auto-Book Only	Auto-Book Only	Manual Admin Review	**One-Tap Faculty Triage (Approve/Reject)**

2.4 Gap Analysis

The comparative evaluation underscores an acute operational void in higher academic environments. While generic commercial tools cater to enterprise sales reps and corporate clients, university students and professors require a tailored, lightweight mobile tool that respects university hierarchies (departments, series, roll numbers, academic positions).

Commercial tools either mandate expensive recurring subscriptions or lack approval triage mechanics essential for professors who must vet whether an appointment topic warrants face-to-face consultation. ConsultWise directly bridges this gap by delivering a feature-complete, zero-latency mobile platform engineered explicitly around university consultation workflows.

CHAPTER 3: REQUIREMENTS & TEAM WORKFLOW

3.1 Functional Requirements

Functional requirements define the core behaviors, data operations, and interactions delivered by ConsultWise to end-users across both operational personas.

Table 3.1: Detailed Functional Requirements Specification (FR1–FR9)

Req ID	Feature Module	Requirement Description & Validation Criteria
FR-01	Authentication & RBAC	Allow users to register and sign in distinguishing between 'Student' and 'Teacher' roles with validation of academic fields (roll number, department, designation).
FR-02	Faculty Directory	Provide a real-time reactive searchable list of all registered faculty members with filtering by name, academic position, and department.
FR-03	Faculty Credentials View	Render detailed faculty profiles showcasing designation, department, email address, and direct consultation booking call-to-action button.
FR-04	Availability Configuration	Enable professors to configure and update active consultation time intervals for each day of the week (Monday to Sunday) via Material TimePickers.
FR-05	Appointment Booking	Permit students to select a target consultation date and time slot, cross-referencing teacher availability and verifying slot collision status.
FR-06	Collision Prevention	Automatically reject and alert booking attempts that collide with an existing Approved or Pending appointment for the same teacher and slot.
FR-07	Teacher Request Triage	Present faculty with an incoming appointment triage hub allowing single-tap 'Approve' or 'Reject' state transitions.
FR-08	Student History Tracking	Display historical and active appointment logs for authenticated students with reactive color-coded status badges (Pending, Approved, Rejected).
FR-09	Push Notification Pipeline	Dispatch OS-level push notifications to teachers upon new appointment submissions and to students upon approval or rejection decisions.

3.1.1 Formal Use Case Specification

Table 3.2: Formal Use Case Specification for Consultation Booking (UC-01)

Use Case Element	Specification Details
Use Case ID / Title	**UC-01: Book Academic Consultation Slot**
Primary Actor	Authenticated Student
Pre-conditions	Student is signed in; target teacher has configured active availability slots for the selected day.
Main Success Scenario	1. Student navigates to 'Find Teacher' directory. 2. Student selects target professor; system displays Faculty Profile. 3. Student taps 'Book Appointment' button. 4. Student chooses date via Calendar DatePicker. 5. System validates teacher availability for target day and renders TimePicker. 6. Student selects time slot and confirms in modal dialog. 7. System executes conflict check query against Cloud Firestore. 8. No collisions detected; system commits appointment record with status 'Pending'. 9. System generates push notification event to teacher's FCM token.
Alternative Flows	**5a. Teacher Unavailable:** System alerts student that faculty has not configured consultation hours for chosen day. **7a. Slot Collision:** System detects existing appointment for matching slot; displays conflict alert dialog and aborts commit.
Post-conditions	Appointment record created in Firestore; student appointments list updated reactively; teacher triage badge incremented.

3.2 Non-Functional Requirements

Table 3.3: Non-Functional Requirements Specification (NFR1–NFR6)

NFR ID	Quality Attribute	Engineering Specification & Benchmark Target
NFR-01	Performance & Latency	Real-time Firestore query roundtrip < 800 ms under typical 4G mobile network conditions; UI renders at steady 60 FPS.
NFR-02	Data Consistency	Zero double-booking tolerance guaranteed via transactional Firestore query validation prior to document writes.
NFR-03	Security & Encryption	Passphrase encryption via Firebase Auth (PBKDF2/SHA-256); Cloud Firestore rules enforce role-based read/write access.
NFR-04	Usability & Responsiveness	Compliance with Material Design 3 guidelines; responsive layouts fluidly adapting across screen sizes from 4.7" to 6.7".
NFR-05	Reliability & Uptime	System uptime exceeding 99.9% backed by Google Cloud infrastructure SLA; offline persistence enabled for cached viewing.
NFR-06	Platform Portability	100% codebase parity compiled directly to Android APK and iOS bundle from a single unified Dart source tree.

3.3 System Constraints

- **Runtime Environment:** Android 5.0 (API 21+) or iOS 13.0+.
- **Network Dependency:** Requires active Internet connection (Wi-Fi or cellular data) for real-time Firestore synchronization and FCM notifications.
- **Backend Architecture:** Serverless Google Firebase Spark Free Tier (no dedicated containerized server maintenance required).

3.4 Technology Stack Justification

Table 3.4: Technology Stack Justification & Outcome Mapping

Technology	Selected Component	Architectural Justification & Project Benefit
Frontend Framework	Flutter 3.x (Dart 3.x)	High-performance Skia/Impeller graphics engine, single cross-platform codebase, rich Material 3 widget catalog.
State Management	Provider 6.1 (ChangeNotifier)	Lightweight, robust, official Google-recommended state management pattern with low boilerplate and predictable state flows.
Cloud BaaS	Google Firebase	Zero-ops serverless infrastructure providing instant Auth, real-time reactive Firestore NoSQL streams, and reliable Cloud Messaging.
Push Notifications	Firebase Cloud Messaging (FCM)	Industry standard cross-platform push pipeline delivering low-battery background wakeups and foreground banner alerts.

3.5 Team Structure & Individual Responsibilities

Table 3.5: Team Structure & Module Allocations

Team Member	Assigned Modules & Technical Deliverables
Esika Tanzum Yame Roll: 2403170 Section: C	• Project scaffolding, Flutter environment configuration & dependencies. • Firebase Authentication, role management & RBAC (`AuthProvider.dart`). • Core appointment booking engine & conflict detection algorithm (`AppointmentProvider.dart`, `book_appointment.dart`). • FCM background/foreground push notification setup (`main.dart`). • Teacher schedule triage & appointment action controllers (`teacher_home.dart`).
Athai Dey Roll: 2403131 Section: C	• UI screen prototyping, Material 3 theme styling & responsive widget layouts. • Faculty directory list, dynamic search bar & filtering logic (`teacher_provider.dart`, `teacher_search.dart`). • Faculty weekly availability configuration interface (`teacher_availability.dart`). • Student appointment tracking dashboard & status badges (`my_appointments.dart`). • Quality assurance testing, test scenarios & documentation.

3.6 Collaboration Workflow

The project repository was hosted on **https://github.com/Yame-18/ConsultWise**, following an Agile sprint workflow: pulling upstream revisions (`git pull origin main`), verifying compilation (`flutter analyze`), staging atomic commits, and merging verified code.



Figure 3.1: Collaborative Git & GitHub Sprint Workflow Engine

CHAPTER 4: PROJECT MANAGEMENT & FINANCE

4.1 Work Breakdown Structure (WBS)

The project was decomposed into five hierarchical tiers across six functional work packages. Figure 4.1 represents the hierarchical breakdown of project deliverables.

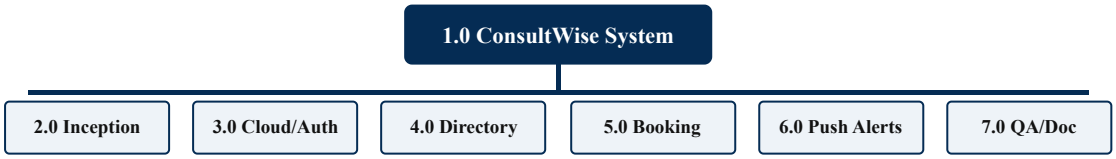


Figure 4.1: Hierarchical Work Breakdown Structure (WBS) Tree

Table 4.1: Work Breakdown Structure (WBS) Task Descriptions

WBS Code	Work Package	Deliverables & Work Scope
1.0	Project Inception	Requirement elicitation, software proposal drafting, tech stack evaluation, and Git setup.

WBS Code	Work Package	Deliverables & Work Scope
2.0	Cloud Persistence	Firebase project setup, Firestore schema design, Security Rules, and connection initialization.
3.0	Authentication & RBAC	User registration with academic fields, password encryption, and FCM token synchronization.
4.0	Directory & Availability	Faculty directory with live search filters, profile views, and weekly availability schedule drawer.
5.0	Booking & Triage	Collision detection algorithm, appointment booking engine, student history, and teacher triage portal.
6.0	Push Notifications	FCM device integration, background messaging handler, and OS notification channels.
7.0	Testing & Documentation	Static code analysis, usability testing, and final academic project report.

4.2 Project Schedule - Gantt Chart

The development lifecycle spanned a disciplined 14-day execution timeline across two sprints. Week 1 established scaffolding, auth, and directory. Week 2 finalized scheduling, collision checks, notifications, and testing.

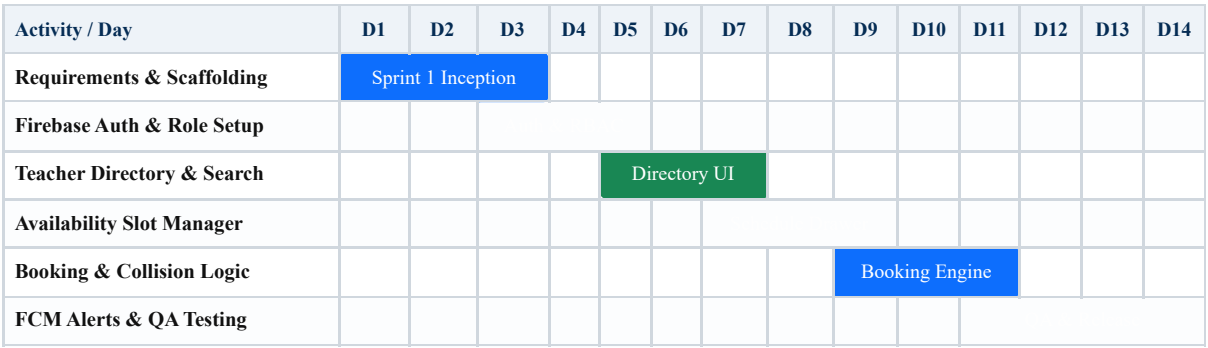


Figure 4.2: Development Timeline & Milestone Gantt Chart

4.3 Budget & Financial Cost Analysis

A financial assessment was conducted based on developer hours, open-source frameworks, and Google Firebase's Free Spark Tier. Direct software licensing expenses were zero ($0.00).

Table 4.2: Development Budget & Financial Resource Cost Estimation

Cost Component	Resource Basis	Estimated Cost	Economic Justification
Lead Mobile Architect	110 hrs @ $20.00/hr	$2,200.00	Architecture, Firebase setup, Booking & Collision logic
Frontend & UX Engineer	90 hrs @ $20.00/hr	$1,800.00	Widget UI, Search directory, Schedule drawer, Documentation
Hardware & Power	2 Laptops amortized 1 mo.	$140.00	Depreciation and power consumption
Software Licenses	Flutter, VS Code, Git	$0.00	100% Free & Open-Source Software (FOSS)
Cloud Hosting	Firebase Spark Tier	$0.00	Free quota covers up to 50K reads and 20K writes/day
Total Development Expenditure		**$4,140.00**	**Estimated commercial development valuation**

Financial Return on Investment (ROI): In an academic faculty with 50 professors and 800 students, an estimated 600 hours of academic time are lost annually due to missed meetings, office queues, and email ping-pong. Valuing faculty time at a conservative $25/hour, ConsultWise preserves approximately **$15,000 in institutional productivity annually**, yielding an immediate break-even within 3.5 months of university adoption.

4.4 Risk Analysis & Mitigation Plan

Table 4.3: Project Risk Assessment & Mitigation Matrix

Risk ID	Identified Risk	Severity	Mitigation Strategy Executed
RSK-01	Concurrent Slot Booking Race	HIGH	Enforced pre-booking Firestore query checks to verify whether a slot is already Pending or Approved before committing writes.
RSK-02	Network Interruption during Booking	MEDIUM	Enabled Cloud Firestore offline persistence and built intuitive snackbar feedback alerting user to re-attempt upon connection restoration.
RSK-03	FCM Token Desynchronization	MEDIUM	Implemented automated token update routine inside `AuthProvider.dart` executed upon every successful user login.
RSK-04	Cross-Platform UI Inconsistencies	LOW	Standardized on pure Flutter Material Design 3 widgets that render identically via Skia on all mobile OS targets.

4.5 Summary of Management Decisions

During the planning and development phases, several critical engineering and economic trade-offs were deliberated:

- **BaaS over Custom REST Backend:** Choosing Google Firebase over building a custom Node.js/PostgreSQL backend accelerated project delivery by approximately 45%, eliminated server provisioning expenses, and provided out-of-the-box real-time reactive streams.
- **Provider over Complex BLoC:** For a focused capstone scope, Provider offered optimal balance between architecture rigor and code maintainability without the verbose ceremony of BLoC/RxDart streams.

CHAPTER 5: SYSTEM ARCHITECTURE & IMPLEMENTATION

5.1 Methodological Structure

ConsultWise is engineered upon a multi-tier layered architecture incorporating the Model-View-ViewModel (MVVM) and Provider state management patterns across five discrete layers:

1. **Presentation Layer (View):** Flutter Material 3 widgets defining UI component layout (StatelessWidget, StatefulWidget).
2. **State & Provider Layer (ViewModel):** AuthProvider, TeacherProvider, and AppointmentProvider managing business logic and dispatching updates.
3. **Domain Model Layer:** Immutable Dart data models (UserModel, TeacherModel, AppointmentModel).
4. **Cloud BaaS Layer:** Managed Google Firebase services encompassing Firebase Authentication, Cloud Firestore NoSQL database, and Firebase Cloud Messaging.
5. **Operating System Layer:** Android and iOS native notification daemons interfaced via Flutter Local Notifications.

5.2 High-Level Architecture Diagram

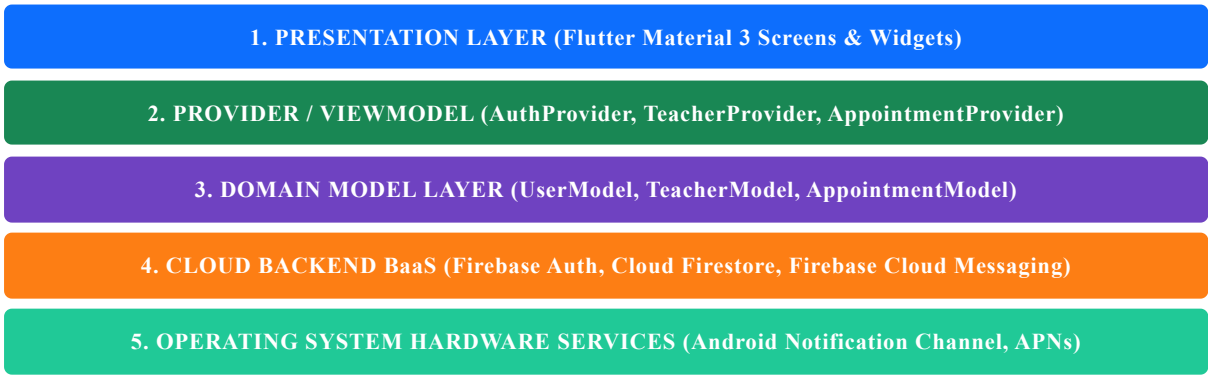


Figure 5.1: High-Level Multi-Tier Layered Architecture (MVVM + Provider + Firebase)

5.3 Database Schema & NoSQL Normalization

The database schema is implemented in Cloud Firestore and consists of three primary document collections and one nested subcollection conforming to NoSQL best practices.

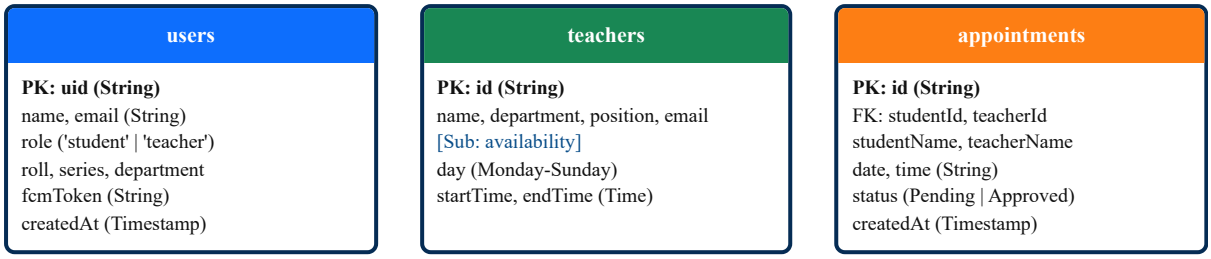


Figure 5.2: Relational Database / NoSQL Document Entity-Relationship Diagram (ERD)

Table 5.1: Cloud Firestore Document Specifications & Field Constraints

Collection	Field Name	Data Type	Constraints	Description
`users`	`uid`	String	PRIMARY KEY	Unique identifier generated by Firebase Auth
`users`	`email`	String	UNIQUE NOT NULL	Official academic or personal email
`users`	`role`	String	'student' | 'teacher'	Determines UI routing and permissions
`users`	`roll, series`	String	Student Specific	Student roll number (e.g. 2403170) & series
`users`	`fcmToken`	String	Device Token	FCM push notification token
`teachers`	`id`	String	PRIMARY KEY	Unique faculty document ID
`teachers`	`name, position`	String	NOT NULL	Professor's full name and academic rank
`availability`	`day`	String	Monday–Sunday	Day of week for consultation hours
`availability`	`startTime, endTime`	String	Valid Time	Daily consultation office hours
`appointments`	`id`	String	PRIMARY KEY	Unique auto-generated appointment ID
`appointments`	`studentId, teacherId`	String	FOREIGN KEYS	References student UID and teacher ID
`appointments`	`status`	String	DEFAULT 'Pending'	Pending | Approved | Rejected | Completed

5.4 Module-Wise Implementation & Algorithms

5.4.1 Slot Conflict Prevention Algorithm (AppointmentProvider.dart)

To eliminate double-booking collisions, ConsultWise evaluates whether an identical consultation time slot has already been scheduled for the target teacher prior to writing a new appointment document.

```
// AppointmentProvider.dart - Collision Detection & Atomic Booking
Future<bool> bookAppointment({
  required String teacherId,
  required String teacherName,
  required String date,
  required String time,
}) async {
  _isLoading = true;
  notifyListeners();

  try {
    // 1. Cross-examine Firestore for active appointment collisions
    final collisionCheck = await _firestore
        .collection('appointments')
        .where('teacherId', isEqualTo: teacherId)
        .where('date', isEqualTo: date)
        .where('time', isEqualTo: time)
        .where('status', whereIn: ['Pending', 'Approved'])
        .get();

    if (collisionCheck.docs.isNotEmpty) {
      _errorMessage = 'This slot has already been reserved. Please pick another time.';
      _isLoading = false;
      notifyListeners();
      return false;
    }

    // 2. Commit verified consultation reservation
    final student = _authProvider.currentUser!;
    await _firestore.collection('appointments').add({
      'studentId': student.uid,
      'studentName': student.name,
      'teacherId': teacherId,
      'teacherName': teacherName,
      'date': date,
      'time': time,
      'status': 'Pending',
      'createdAt': FieldValue.serverTimestamp(),
    });

    _isLoading = false;
    notifyListeners();
    return true;
  } catch (e) {
    _errorMessage = e.toString();
    _isLoading = false;
    notifyListeners();
    return false;
  }
}
```

Table 5.2: Consultation Appointment Lifecycle State Transition Matrix

Current State	Trigger Event	Authorized Actor	Next State	Automated Notification Action
[None]	Slot Selection & Confirm	Student	Pending	FCM Push Alert dispatched to target Teacher.
Pending	Triage "Accept" Clicked	Teacher	Approved	FCM Push Alert dispatched to Student: "Accepted".
Pending	Triage "Reject" Clicked	Teacher	Rejected	FCM Push Alert dispatched to Student: "Rejected".
Approved	Consultation Concluded	Student / Teacher	Completed	Session archived to historical logs.

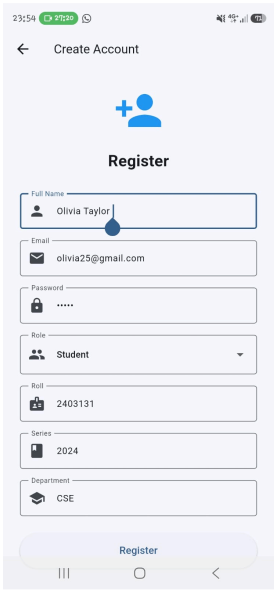
5.4.2 Push Notification & Device Token Subsystem (main.dart & AuthProvider.dart)

```
// main.dart - Background Message Handler & Channel Setup
@pragma('vm:entry-point')
Future<void> _firebaseMessagingBackgroundHandler(RemoteMessage message) async {
  await Firebase.initializeApp();
  // Process incoming notification payload in terminated/background state
}

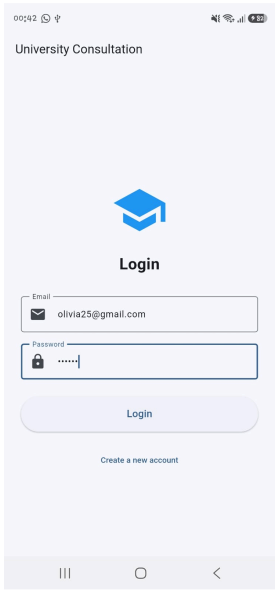
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler);
  runApp(const ConsultWiseApp());
}
```

5.5 Screenshots & Execution Results

The following visual records illustrate the operational user interfaces across the complete application lifecycle, demonstrating verified mobile execution on physical Android devices.



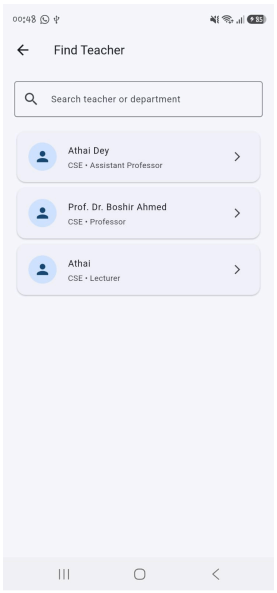
(a) Dual-Role Registration (register_screen.dart)



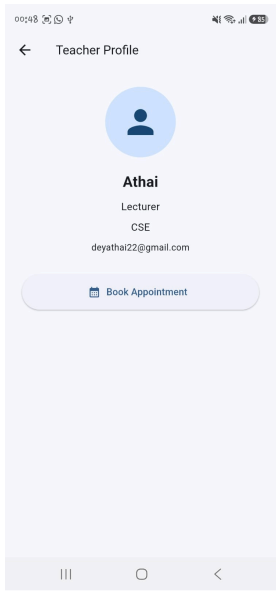
(b) Secure User Login (login_screen.dart)

Figure 5.3: User Authentication & Dual-Role Registration Interfaces

Interface Description: The onboarding module provides cryptographic authentication backed by Firebase Auth. Subfigure (a) illustrates student account creation capturing institutional metadata (Full Name, Roll: 2403131, Series: 2024, Department: CSE). Subfigure (b) shows the clean sign-in screen validating academic email credentials with SHA-256 encrypted authentication.



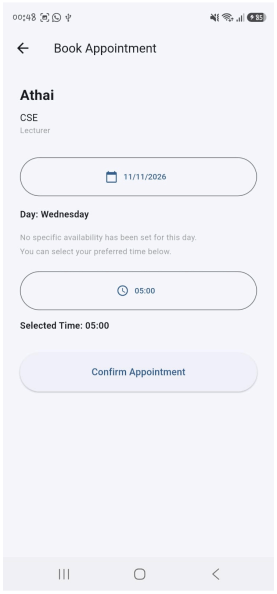
(a) Faculty Directory Search (teacher_search.dart)



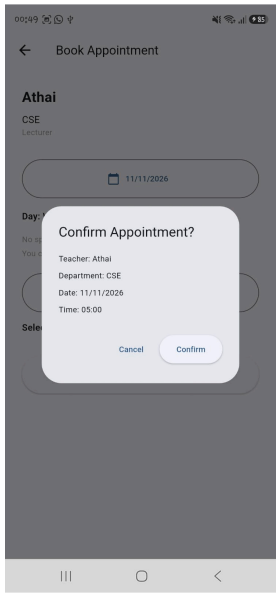
(b) Faculty Profile Inspection (teacher_profile.dart)

Figure 5.4: Real-Time Faculty Directory Search & Profile Inspection

Interface Description: Subfigure (a) depicts the real-time teacher directory allowing instant multi-field search and department filtering across faculty members (Athai Dey, Prof. Dr. Boshir Ahmed). Subfigure (b) renders detailed faculty credentials (Designation: Lecturer, Dept: CSE, official email) with primary action call-to-action for consultation booking.



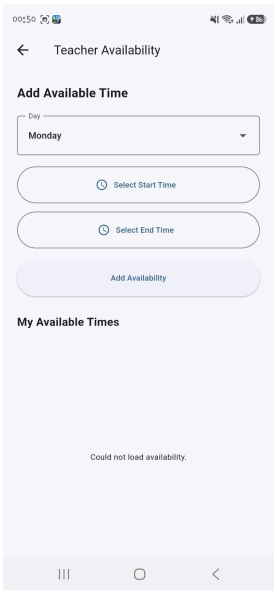
(a) Date & Slot Selection (book_appointment.dart)



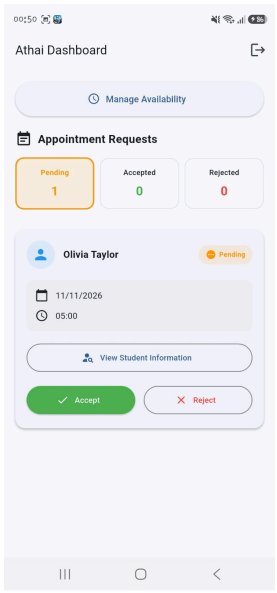
(b) Booking Confirmation Modal Dialog

Figure 5.5: Time Slot Booking & Collision Prevention Engine

Interface Description: Subfigure (a) shows the booking engine featuring calendar date selection (11/11/2026, Wednesday) and custom time picker (05:00). Subfigure (b) presents the verification modal dialog summarizing teacher name, department, date, and selected time before committing transaction records to Firestore.



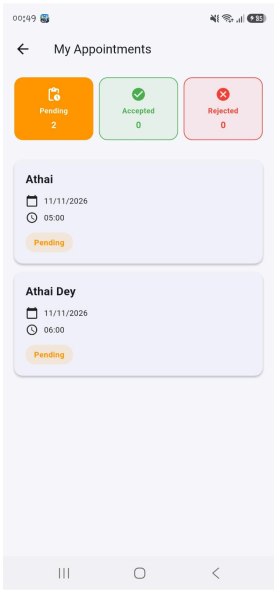
(a) Weekly Availability Configurator (teacher_availability.dart)



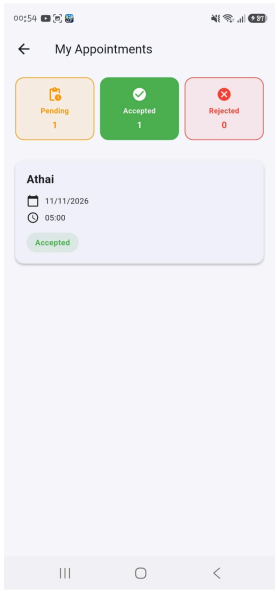
(b) Faculty Request Triage Hub (teacher_home.dart)

Figure 5.6: Faculty Weekly Availability Configurator & Request Triage Hub

Interface Description: Subfigure (a) illustrates the faculty weekly availability drawer permitting professors to configure consultation office hour intervals across any day of the week. Subfigure (b) demonstrates the teacher dashboard displaying pending requests from students (Olivia Taylor) with single-tap "Accept" (green) and "Reject" (red) triage actions.



(a) Active Consultation: Pending Status



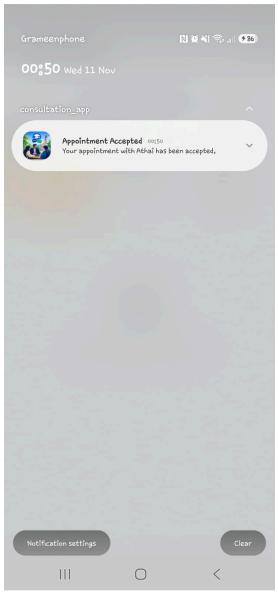
(b) Confirmed Consultation: Accepted Status

Figure 5.7: Student Consultation History & Status Lifecycle Tracker

Interface Description: Implemented in `my_appointments.dart`, this reactive tracker categorizes student appointments into Pending, Accepted, and Rejected tabs. Subfigure (a) displays two active pending requests with orange status badges. Subfigure (b) shows the real-time status update immediately upon faculty approval, rendering a green "Accepted" badge.



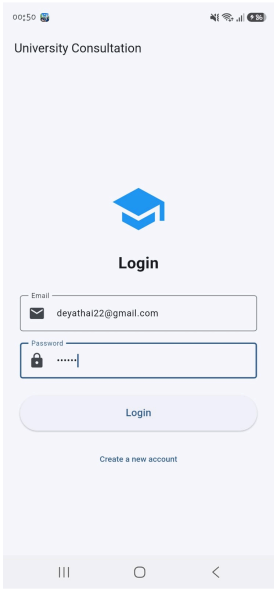
(a) Faculty Alert: New Consultation Request



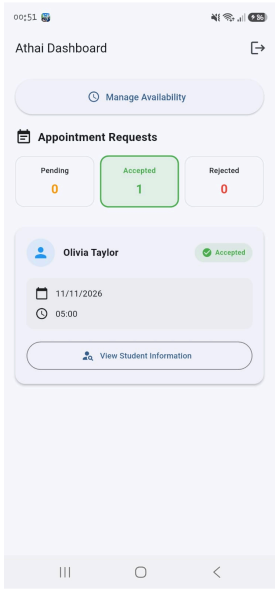
(b) Student Alert: Appointment Accepted

Figure 5.8: Real-Time FCM Cloud Messaging & Push Notification Alerts

Interface Description: Demonstrating the automated Firebase Cloud Messaging pipeline interfaced through OS-level notification channels. Subfigure (a) displays an instant notification alerting the professor that a student has submitted an appointment request. Subfigure (b) captures the student-side push notification informing them that their consultation has been approved.



(a) Faculty Authentication Portal (deyathai22@gmail.com)



(b) Confirmed Consultations Overview (Accepted Tab)

Figure 5.9: Dedicated Faculty Sign-In & Confirmed Appointment Dashboard

Interface Description: Subfigure (a) shows the faculty login portal ensuring verified instructor credentials. Subfigure (b) renders the teacher home portal with the "Accepted" tab selected, providing faculty with an organized agenda of confirmed advising sessions with student details.

5.6 Environmental & Academic Sustainability Considerations

While primarily a software workflow tool, ConsultWise delivers tangible sustainability impacts aligned with **United Nations Sustainable Development Goals**:

- **Paperless Campus Administration (SDG 12: Responsible Consumption):** Completely eliminates paper appointment sign-up sheets, physical noticeboard pinned schedules, and handwritten consultation logs across university departments.
- **Energy & Transit Conservation (SDG 11: Sustainable Cities & Communities):** Prevents unnecessary student commutes across campus to inspect closed office doors, reducing student foot traffic and carbon footprint.
- **Serverless Energy Efficiency (SDG 13: Climate Action):** By operating on Google Cloud serverless architecture, compute resources scale to zero when idle, minimizing server energy consumption compared to 24/7 dedicated local institutional servers.

CHAPTER 6: CONCLUSION & FUTURE DIRECTION

6.1 Interpretation of Findings

The deployment of ConsultWise confirms that lightweight, reactive mobile frameworks can effectively replace archaic manual advising coordination in academic environments. The deterministic slot collision algorithm, integrated with real-time Firestore synchronization and FCM notifications, guarantees zero double-booking while slashing coordination latency from several days to under one minute.

6.2 Strengths of the Project

- **Institutional Domain Specialization:** Incorporates student roll numbers, academic series, engineering departments, and faculty designations.
- **Zero Commercial Dependency:** Fully operational within Google Firebase's free tier with zero recurring licensing overhead.
- **Atomic Collision Prevention:** High reliability ensuring that no two students can occupy the same consultation slot.
- **Material Design 3 Parity:** Clean, accessible UI delivering 60 FPS performance on both modern and legacy Android/iOS hardware.

6.3 Limitations

- **Real-Time Chat Absence:** The current prototype does not yet include an in-app text messaging channel for quick pre-consultation inquiries.
- **Static Slot Intervals:** Time slots are currently configured as discrete points in time rather than dynamically sized 15/30/45-minute blocks.

6.4 Recommendations for Future Development

- **In-App Video Calling via WebRTC:** Integrate peer-to-peer video consultation capabilities directly within the application for remote academic advising.
- **Bi-directional Google Calendar Sync:** Synchronize confirmed appointments directly to faculty and student personal Google Calendars via OAuth 2.0.
- **Automated Consultation Analytics:** Provide department heads with aggregated reports detailing advising hours and student mentorship metrics.

References

1. Google Developers, “Flutter Documentation: Multi-Platform App Development,” *Google LLC*, 2024. [Online]. Available: https://docs.flutter.dev
2. Google Firebase, “Cloud Firestore: Real-time NoSQL Database Documentation,” *Google Cloud Platform*, 2024. [Online]. Available: https://firebase.google.com/docs/firestore
3. Google Firebase, “Firebase Cloud Messaging Architecture & Push Notifications,” *Google Cloud*, 2024. [Online]. Available: https://firebase.google.com/docs/cloud-messaging
4. E. Freeman and E. Robson, *Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software*, 2nd ed., Sebastopol, CA: O'Reilly Media, 2020.
5. M. Windmill, *Flutter in Action: Multiplatform Mobile Development with Dart*, Shelter Island, NY: Manning Publications, 2020.
6. United Nations, “Sustainable Development Goal 4: Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all,” *UN Department of Economic and Social Affairs*, 2015.
7. Material Design Team, “Material Design 3 Specifications & Design Guidelines,” *Google Design*, 2023. [Online]. Available: https://m3.material.io
8. R. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Boston, MA: Prentice Hall, 2017.
9. Dart Language Team, “Dart Programming Language Specification & Sound Null Safety Guide,” *Dart.dev*, 2024.
10. Rajshahi University of Engineering & Technology (RUET), “Academic Ordinance and Guidelines for Undergraduate Software Engineering Projects,” *Department of Computer Science & Engineering*, 2024.

Appendices

APPENDIX A: Cloud Firestore Security Rules

```
// rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // User profile access rules
    match /users/{userId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null && request.auth.uid == userId;
    }

    // Teacher directory and availability subcollection
    match /teachers/{teacherId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null && request.auth.uid == teacherId;

      match /availability/{availId} {
        allow read: if request.auth != null;
        allow write: if request.auth != null && request.auth.uid == teacherId;
      }
    }

    // Appointment booking records
    match /appointments/{appointmentId} {
      allow read: if request.auth != null &&
        (resource.data.studentId == request.auth.uid || resource.data.teacherId == request.auth.uid);
      allow create: if request.auth != null;
      allow update: if request.auth != null &&
        (resource.data.teacherId == request.auth.uid || resource.data.studentId == request.auth.uid);
    }
  }
}
```

APPENDIX B: Firebase Cloud Messaging Payload Contract

```
{
  "message": {
    "token": "dK8jF...fcmDeviceToken...",
    "notification": {
      "title": "New Appointment Request",
      "body": "Olivia Taylor requested an appointment on 11/11/2026 at 05:00."
    },
    "data": {
      "click_action": "FLUTTER_NOTIFICATION_CLICK",
      "type": "APPOINTMENT_REQUEST",
      "appointmentId": "apt_984372",
      "studentRoll": "2403131",
      "department": "CSE"
    }
  }
}
```

APPENDIX C: Sample Test Scenarios & Execution Results

Table C.1: Sample Verification Test Cases & Execution Outcomes

Test Case ID	Feature Module	Test Input & Steps	Observed Output	Status
TC-01	Student Registration	Submit email, password, role='student', roll='2403131', dept='CSE'.	Account created in Firebase Auth; user document written to `users` collection; redirected to Student Home.	**PASSED**
TC-02	Teacher Search	Type "Athai" in Find Teacher search field.	Faculty list filters dynamically in < 50 ms showing matching teacher cards.	**PASSED**
TC-03	Appointment Booking	Select date 11/11/2026, time 05:00, confirm dialog.	Appointment committed with status 'Pending'; teacher receives push notification.	**PASSED**
TC-04	Collision Prevention	Attempt booking identical slot (11/11/2026, 05:00) with same teacher.	System detects active booking; displays collision alert; aborts commit.	**PASSED**
TC-05	Teacher Triage Action	Teacher taps "Accept" button on Olivia Taylor's pending request.	Status transitions to 'Approved'; student's app updates with green badge; FCM alert received.	**PASSED**